
UWB Indoor Positioning

A Plain-English Guide to Every Equation

Companion to the technical design document.

No prior engineering or maths background assumed.

Hardware Makerfabs ESP32 UWB Pro (Decawave DW1000)
Goal Find the XYZ position and orientation of a moving tag
Version 1.0 — June 2026

Contents

1	The Big Picture in One Page	2
2	Step 1 — Measuring Distance with Radio Pulses (DS-TWR)	2
2.1	Why Timing is Hard — The Clock Problem	2
2.2	The Four-Message Fix	3
2.3	Building the Four Timing Intervals	3
2.4	The DS-TWR Formula	3
2.4.1	Why does this formula work? (Step-by-step)	4
2.4.2	How the Clock Error Cancels	4
2.4.3	Converting Travel Time to Distance	4
2.5	Antenna Delay Calibration	5
3	Step 2 — Detecting Blocked Paths (NLOS)	5
3.1	What the Chip Reports	6
3.2	Computing First-Path Power	6
3.3	The NLOS Score	6
3.4	Correcting the NLOS Range Error	7
4	Step 3 — Finding Position from Distances (Multilateration)	7
4.1	The Measurement Equation	8
4.2	Minimising the Total Error	8
4.3	Solving It Iteratively — Levenberg-Marquardt (LM)	9
4.4	Throwing Out Bad Anchors (MAD Gating)	9
4.5	How Certain Are We? (Position Covariance)	10
4.6	Solving for a Timing Bias Too (GPS-Style)	10
5	Step 4 — Smoothing Position Over Time (EKF / MHE)	11
5.1	The Extended Kalman Filter (EKF)	11
5.1.1	The State Vector	11
5.1.2	Step A: Predict (Physics)	11
5.1.3	Step B: Update (Measurement)	12
5.2	Moving Horizon Estimator (MHE)	13
6	Step 5 — Adding Orientation (IMU Fusion)	14
6.1	Describing Orientation — Why Quaternions?	14
6.2	The Rotation Matrix	15
6.3	How Orientation Updates (Quaternion Kinematics)	15
6.4	Position Propagation from IMU	15
6.5	Extracting Roll, Pitch, Yaw from a Quaternion	16
7	Step 6 — Finding Anchor Positions Automatically (MDS)	16
7.1	Collecting Pairwise Distances	17
7.2	Step 1: The Squared-Distance Matrix	17
7.3	Step 2: Double-Centering	17
7.4	Step 3: Eigendecomposition	17
7.5	Step 4: Fix the Coordinate Frame	18
7.6	How Accurate Is It?	18
8	How All the Pieces Fit Together	19

1 The Big Picture in One Page

We are building an indoor GPS. Four small radio modules (called **anchors**) are fixed at known positions around a room. One more module (called the **tag**) is carried by a person or robot. The tag measures how far it is from each anchor by timing how long a radio pulse takes to travel between them. A laptop then uses those four distances to calculate where the tag must be.

That is the whole idea. Everything else in this document is about doing it more accurately.

Why not just use GPS? Ordinary GPS works by receiving signals from satellites 20 000 km above Earth. Indoors, those signals are too weak and reflect off walls, making them unreliable. Our system places the equivalent of “mini-satellites” in the same room, which solves both problems.

The processing chain — each step in the pipeline is covered by one section of this document:

Step	What happens
1	Tag sends a radio pulse to each anchor and times the round-trip (§2)
2	We check whether any anchor’s signal was blocked or reflected (§3)
3	We use the four distances to triangulate position (§4)
4	We smooth the position over time using a physics model (§5)
5	We add orientation (tilt, heading) from an IMU sensor (§6)
6	Anchors measure each other to find their own positions automatically (§7)

2 Step 1 — Measuring Distance with Radio Pulses (DS-TWR)

DS-TWR stands for **Double-Sided Two-Way Ranging**. It is the method used to measure how far the tag is from one anchor. The core idea: send a radio pulse from the tag to the anchor, measure how long it takes, and multiply by the speed of light to get a distance. Then correct for the fact that neither device has a perfect clock.

2.1 Why Timing is Hard — The Clock Problem

Radio pulses travel at the speed of light: $c = 3 \times 10^8$ metres per second. At 3 metres distance, the travel time is:

$$\text{time} = \frac{\text{distance}}{\text{speed}} = \frac{3}{3 \times 10^8} = 10 \text{ nanoseconds} \quad (\text{a nanosecond is } 10^{-9} \text{ seconds})$$

So we need to measure 10 nanoseconds accurately. That is extremely short. The clock inside a cheap crystal oscillator runs at a slightly wrong speed — typically off by about 20 parts per million (ppm). What does that mean?

“20 parts per million” means the clock gains or loses 20 microseconds for every second it runs. If the tag waits 1 millisecond (1/1000 of a second) before sending its reply, the

error in that delay is:

$$20 \times 10^{-6} \times 1 \times 10^{-3} \text{ seconds} = 20 \text{ nanoseconds of error}$$

Since light travels 3×10^8 m/s, 20 ns of timing error corresponds to:

$$20 \times 10^{-9} \times 3 \times 10^8 = 6 \text{ metres of fake distance}$$

A 6 metre error from a 1 ms processing delay! This is why we cannot simply time a single pulse.

2.2 The Four-Message Fix

DS-TWR uses **four messages** between the tag (T) and anchor (A) so that the clock errors cancel out mathematically:

Msg	Who sends	Name	What time is recorded
1	Tag → Anchor	POLL	Tag records send time t_1 ; Anchor records receive time t_2
2	Anchor → Tag	POLL-ACK	Anchor records send time t_3 ; Tag records receive time t_4
3	Tag → Anchor	RANGE	Tag records send time t_5 ; also sends t_1, t_4, t_5 inside the message
4	Anchor → Tag	REPORT	Anchor records receive time t_6 ; computes and sends back distance

Think of it like a carefully timed conversation where both sides know exactly when they spoke and when they heard a reply.

2.3 Building the Four Timing Intervals

From those six timestamps, we build four intervals. Each is measured in **clock ticks** (one tick on the DW1000 chip = 15.65 picoseconds, i.e. 15.65×10^{-12} seconds):

$R_a = t_4 - t_1$	Tag's round-trip time	(tag sent POLL at t_1 , received ACK at t_4)
$D_a = t_3 - t_2$	Anchor's reply delay	(anchor received POLL at t_2 , sent ACK at t_3)
$R_b = t_6 - t_3$	Anchor's round-trip time	(anchor sent ACK at t_3 , received RANGE at t_6)
$D_b = t_5 - t_4$	Tag's reply delay	(tag received ACK at t_4 , sent RANGE at t_5)

R stands for “Round-trip.” D stands for “Delay” (the waiting time before replying). The subscripts a and b indicate which side (tag or anchor) did the measuring.

If the clocks were perfect and the one-way travel time is τ :

$$R_a = \underbrace{2\tau}_{\text{signal went there and back}} + \underbrace{D_a}_{\text{anchor's delay}} \quad \text{and} \quad R_b = 2\tau + D_b$$

2.4 The DS-TWR Formula

$$\tau_{\text{ToF}} = \frac{R_a \cdot R_b - D_a \cdot D_b}{R_a + R_b + D_a + D_b} \quad (1)$$

τ_{ToF}	The one-way travel time we want (ToF = Time of Flight)
R_a, R_b	The two round-trip measurements
D_a, D_b	The two processing delays

2.4.1 Why does this formula work? (Step-by-step)

Step 1. In the ideal case, substitute $R_a = 2\tau + D_a$ and $R_b = 2\tau + D_b$ into the formula:

$$\tau_{\text{ToF}} = \frac{(2\tau + D_a)(2\tau + D_b) - D_a D_b}{(2\tau + D_a) + (2\tau + D_b) + D_a + D_b}$$

Step 2. Expand the numerator:

$$(2\tau + D_a)(2\tau + D_b) - D_a D_b = 4\tau^2 + 2\tau D_b + 2\tau D_a + D_a D_b - D_a D_b = 4\tau^2 + 2\tau(D_a + D_b)$$

Step 3. Simplify the denominator:

$$(2\tau + D_a) + (2\tau + D_b) + D_a + D_b = 4\tau + 2D_a + 2D_b = 2(2\tau + D_a + D_b)$$

Step 4. Divide:

$$\tau_{\text{ToF}} = \frac{4\tau^2 + 2\tau(D_a + D_b)}{2(2\tau + D_a + D_b)} = \frac{2\tau(2\tau + D_a + D_b)}{2(2\tau + D_a + D_b)} = \tau \checkmark$$

The delays D_a and D_b cancel out completely. The formula returns the exact travel time regardless of how long the devices waited before replying.

2.4.2 How the Clock Error Cancels

With real clocks, the tag's clock runs at a slightly wrong speed. Let ε_T be how wrong (e.g. $\varepsilon_T = 20 \times 10^{-6}$ for a 20 ppm error). Every interval measured by the tag is stretched by $(1 + \varepsilon_T)$. Similarly the anchor's clock error is ε_A .

So the actual measured values are:

$$R_a = (2\tau + D_a)(1 + \varepsilon_T) \quad R_b = (2\tau + D_b)(1 + \varepsilon_A)$$

When you substitute these into the DS-TWR formula and do the algebra, the ε_T and ε_A terms cancel to first order, leaving:

$$\tau_{\text{ToF}} \approx \tau + \underbrace{(\text{error proportional to } \varepsilon^2)}_{\approx (20 \text{ ppm})^2 = 4 \times 10^{-10}, \text{ i.e. less than } 0.1 \text{ mm}}$$

The remaining error is tiny (less than 0.1 mm), no matter how long the devices waited.

2.4.3 Converting Travel Time to Distance

$$\hat{d} = c \cdot \tau_{\text{ToF}} + b_{\text{ant}} \tag{2}$$

$\hat{d}$	Measured distance (the hat ^ means “estimated”, not exact)
c	Speed of light: 2.998×10^8 m/s
τ_{ToF}	Travel time from equation 1
b_{ant}	A small correction for each board’s internal wiring delay (see below)

A tag is 3 metres from an anchor. The DW1000 clock ticks at 64 GHz (i.e. $1/64 \times 10^9$ seconds per tick = 15.65 ps per tick). The one-way travel time is 10 ns = 639 ticks. Say the anchor delays 1 ms ($D_a \approx 63,900,000$ ticks) and the tag delays 1.2 ms ($D_b \approx 76,700,000$ ticks). Then:

$$R_a = 2 \times 639 + 63,900,000 = 63,901,278 \text{ ticks}$$

$$R_b = 2 \times 639 + 76,700,000 = 76,701,278 \text{ ticks}$$

Plugging into equation 1: the huge delay numbers cancel in numerator and denominator, and we get back ≈ 639 ticks. Multiplying: $639 \times 15.65 \text{ ps} \times 3 \times 10^8 \text{ m/s} = 3.0 \text{ m}$. ✓

2.5 Antenna Delay Calibration

Every DW1000 chip has a slightly different internal wiring delay between its digital clock and its physical antenna. This shifts every distance reading by the same constant amount. Calibration finds and removes that constant.

Think of it like zeroing a scale before weighing something — the scale might read 2 grams when empty, so you subtract 2 grams from every reading.

The DW1000 has a register (Δ , a 16-bit number) that shifts the receive timestamp. We find the best value by trying different Δ values and comparing the measured distance against a known true distance d^* :

$$\Delta^* = \arg \min_{\Delta} \left(\mathbb{E}[\hat{d}(\Delta)] - d^* \right)^2$$

Plain English: “Find the Δ that makes the average measured distance $\mathbb{E}[\hat{d}(\Delta)]$ as close as possible to the true distance d^* .” The symbol $\arg \min$ means “the value of Δ that minimises the expression in the brackets.” We use a **binary search** (the same idea as guessing a number between 1 and 100 by always trying the middle of the remaining range), which finds the answer in 14 steps.

After calibration, the distance error caused by internal chip delays is reduced to under 1.4 mm.

3 Step 2 — Detecting Blocked Paths (NLOS)

NLOS stands for **Non-Line-of-Sight**. It means the direct path between tag and anchor is blocked by a wall, person, or furniture. When this happens, the radio pulse still arrives at the anchor — but only after bouncing off other surfaces. Because the bounced path is longer than the direct path, the measured distance is always *too large*. We never get a negative NLOS error, only positive ones.

The DW1000 chip can tell us whether this happened by reporting how much of the received energy came in early (the direct path) versus late (via reflections).

Shout “hello” across a room. In a normal room you hear one clear echo. Now put a wall in the way — some sound diffracts around it, and some bounces off side walls, arriving a little later and sounding “muddy.” More spread-out echoes = more reflected paths = NLOS.

3.1 What the Chip Reports

After every received message, the DW1000 automatically fills three registers:

Register	What it stores	In plain words
FP_AMPL1, 2, 3	Three amplitude samples	Strength of the <i>first-arriving</i> signal
RXPACC	Preamble accumulator count	How many samples were averaged (normalisation factor)
CIR_PWR	Total channel power	Strength of <i>everything</i> received, including reflections

3.2 Computing First-Path Power

We combine the three first-path amplitudes A_1, A_2, A_3 into a single power value. Power is proportional to amplitude squared, so we square each and add them. Then we normalise by N_{acc}^2 (where $N_{\text{acc}} = \text{RXPACC}$) and convert to decibels (dB):

$$P_{\text{FP}} = 10 \log_{10} \left(\frac{A_1^2 + A_2^2 + A_3^2}{N_{\text{acc}}^2} \right) - A_{\text{const}} \quad (3)$$

P_{FP}	First-path power in dBm (power of the direct-arrival signal)
A_1, A_2, A_3	The three FP_AMPL register values
N_{acc}	RXPACC — normalises for how many preamble samples were collected
A_{const}	Chip constant: 121.74 dBm at 16 MHz PRF; 113.77 dBm at 64 MHz PRF
$\log_{10}$	Logarithm base 10 (tells us the power of 10 needed to reach a number)

A decibel is a way of expressing a ratio on a squished scale, because power can vary by enormous factors. $10 \log_{10}(x)$ converts a power ratio x into dB. Key facts:

- +3 dB means the power doubled ($10^{0.3} \approx 2$).
- +10 dB means the power became 10 times larger ($10^1 = 10$).
- −10 dB means the power became 10 times smaller.

We use dB because “6 dB difference” is easier to write than “the power ratio is 3.98”.

3.3 The NLOS Score

The total received power P_{RX} is read from CIR_PWR by the chip’s driver. The NLOS score is simply the gap between total and first-path power:

$$\Delta P_{\text{NLOS}} = P_{\text{RX}} - P_{\text{FP}} \quad (4)$$

ΔP_{NLOS}	NLOS score in dB. Δ (delta) means “difference.”
P_{RX}	Total received power (first path + all reflections)
P_{FP}	First-path power (direct arrival only)

How to read the score:

NLOS score	Meaning	What we do
Less than 3 dB	Most energy came in early (LOS)	Trust this anchor fully
3 to 6 dB	Some late energy (soft NLOS)	Reduce this anchor’s weight
More than 6 dB	Most energy came in late (hard NLOS)	Strongly distrust or ignore

3.4 Correcting the NLOS Range Error

Because the blocked path is longer than the direct path, the measured distance is always too large. We estimate how much to subtract:

$$b_{\text{NLOS}} \approx k_{\text{NLOS}} \cdot \max(0, \Delta P_{\text{NLOS}} - 3 \text{ dB}) \tag{5}$$

b_{NLOS}	The distance correction to subtract from $\hat{d}$ (in metres)
k_{NLOS}	A tunable constant, typically 0.02–0.05 m per dB
$\max(0, \dots)$	“Take the larger of 0 and the expression.” If the score is below 3 dB, the correction is zero (no correction needed in LOS).

NLOS score = 8 dB. Correction: $b = 0.03 \times (8 - 3) = 0.15$ m. We subtract 15 cm from the measured range before using it in the position solver.

NLOS detection is free — the chip computes it from data it already collects. No extra hardware required. This is the first open-source UWB system to use it.

4 Step 3 — Finding Position from Distances (Multilateration)

“Trilateration” or “multilateration” means finding a position given distances to several known points. You have distances from the tag to each anchor. Each distance says “I could be anywhere on a circle (or sphere) of that radius around this anchor.” Multiple circles narrow it down to one intersection point — your position.

Imagine you can hear three radio stations and you know how strong each signal is (weaker = farther away). You draw a circle of the right radius around each station on a map. The point where all three circles cross is where you are. With four anchors in 3D (one per spatial dimension plus one spare), you can find position in XYZ.

4.1 The Measurement Equation

For each anchor i (where i goes from 1 to N , the number of anchors):

$$\hat{d}_i = \underbrace{\|\mathbf{x} - \mathbf{a}_i\|}_{\text{true distance}} + \underbrace{e_i}_{\text{noise}} \quad (6)$$

$\hat{d}_i$	Measured (noisy) distance to anchor i
$\mathbf{x}$	The tag's position we are trying to find — a vector like $[x, y]$ or $[x, y, z]$
$\mathbf{a}_i$	The known position of anchor i — also a vector like $[a_x, a_y]$
$\ \mathbf{x} - \mathbf{a}_i\ $	The straight-line distance from tag to anchor i This equals $\sqrt{(x - a_x)^2 + (y - a_y)^2}$ in 2D (Pythagoras' theorem)
e_i	Measurement noise — a random error, assumed to follow a bell curve
$\mathcal{N}(0, \sigma_i^2)$	Bell curve (Gaussian) with average 0 and spread σ_i
σ_i	Standard deviation: roughly the expected size of the error

The noise σ_i for each anchor depends on its NLOS score:

$$\sigma_i = \sigma_{\text{base}} \cdot 10^{\Delta P_{\text{NLOS},i} / 20} \quad (7)$$

σ_{base}	Baseline noise (e.g. 0.10 m) when the anchor is in perfect LOS
$10^{\Delta P / 20}$	A multiplier > 1 that grows with the NLOS score At 10 dB NLOS: $10^{10/20} = 10^{0.5} \approx 3.16$, so $\sigma_i \approx 3.16 \times \sigma_{\text{base}}$

An NLOS anchor gets a larger σ_i , which means we trust it less (see weights below).

4.2 Minimising the Total Error

We want to find the position $\mathbf{x}$ that makes all the distance equations as consistent as possible. The **cost function** is:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \sum_{i=1}^N w_i \left(\|\mathbf{x} - \mathbf{a}_i\| - \hat{d}_i \right)^2, \quad w_i = \frac{1}{\sigma_i^2} \quad (8)$$

$\arg \min_{\mathbf{x}}$	“Find the $\mathbf{x}$ that makes the following expression as small as possible”
$\sum_{i=1}^N$	Add up the following term for each anchor from $i = 1$ to $i = N$
w_i	Weight for anchor i : larger weight = we trust this anchor more
$w_i = 1/\sigma_i^2$	Noisier anchors (large σ_i) get smaller weight (large σ_i^2 in denominator)
$(\ \mathbf{x} - \mathbf{a}_i\ - \hat{d}_i)^2$	Squared difference between the distance our guess implies and what we actually measured to anchor i

We are guessing a position $\mathbf{x}$. For each anchor, we check: “if I were at position $\mathbf{x}$, what distance would I expect to anchor i ? How far is that from what I actually measured?” We square that gap so positive and negative errors don't cancel, multiply by how much we trust the anchor, and add it all up. We adjust our guess to make this total as small as possible.

4.3 Solving It Iteratively — Levenberg-Marquardt (LM)

This minimisation has no closed-form solution (we cannot just rearrange the equation). So we use an iterative algorithm called **Levenberg-Marquardt** that makes small improvements to the guess until it stops improving.

The residual vector. For a current guess $\mathbf{x}_k$, define the residual for anchor i as:

$$r_i(\mathbf{x}_k) = \|\mathbf{x}_k - \mathbf{a}_i\| - \hat{d}_i$$

This is simply: (distance my guess implies) – (what I measured). If $r_i = 0$, the guess is perfect for anchor i . If $r_i > 0$, the guess is too far from anchor i .

The Jacobian. To improve the guess, we need to know: “which direction should I move $\mathbf{x}$ to reduce each r_i most quickly?” The answer is the **Jacobian** matrix $\mathbf{J}$. Each row i of $\mathbf{J}$ is:

$$\mathbf{J}_i = \frac{\partial r_i}{\partial \mathbf{x}} = \frac{(\mathbf{x} - \mathbf{a}_i)^\top}{\|\mathbf{x} - \mathbf{a}_i\|} \quad (9)$$

$\partial r_i / \partial \mathbf{x}$	“How much does r_i change when I nudge $\mathbf{x}$ a tiny bit?” (partial derivative)
$(\mathbf{x} - \mathbf{a}_i)$	The vector pointing from anchor i to the current guess
$\ \mathbf{x} - \mathbf{a}_i\ $	The length of that vector
$(\mathbf{x} - \mathbf{a}_i) / \ \mathbf{x} - \mathbf{a}_i\ $	This divides by the length to get a <i>unit vector</i> (length = 1)

In plain words: $\mathbf{J}_i$ is a unit vector pointing from anchor i toward the current position guess. Moving the guess in that direction increases the distance to anchor i by exactly 1 metre per metre of movement. This is just geometry.

The update step. Given the Jacobian $\mathbf{J}$, the weight matrix $\mathbf{W} = \text{diag}(w_1, \dots, w_N)$ (a matrix with weights on the diagonal, zeros elsewhere), and the damping parameter λ , the new guess is:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left(\mathbf{J}^\top \mathbf{W} \mathbf{J} + \lambda \text{diag}(\mathbf{J}^\top \mathbf{W} \mathbf{J}) \right)^{-1} \mathbf{J}^\top \mathbf{W} \mathbf{r}(\mathbf{x}_k) \quad (10)$$

This looks intimidating but the logic is:

1. $\mathbf{J}^\top \mathbf{W} \mathbf{r}$ — the weighted gradient (which direction is downhill?).
2. $(\mathbf{J}^\top \mathbf{W} \mathbf{J})^{-1}$ — scales the step to the right size.
3. λ — a safety dial. Large λ : take a tiny, safe step (slow but won’t overshoot). Small λ : take a big, efficient step (fast near the solution). The algorithm sets λ automatically: increase it if the step made things worse; decrease it if the step helped.

The algorithm typically converges in 5–15 iterations.

4.4 Throwing Out Bad Anchors (MAD Gating)

After the first solve, we check whether any anchor is wildly inconsistent (e.g. NLOS blocked and the correction wasn’t enough). We use the **Median Absolute Deviation**:

$$\hat{\sigma}_{\text{robust}} = 1.4826 \cdot \text{median}(|r_i - \text{median}(\mathbf{r})|) \quad (11)$$

$\text{median}(\mathbf{r})$	The middle value of all residuals when sorted smallest to largest
$ r_i - \text{median}(\mathbf{r}) $	How far each residual is from the median
$\text{median}(\dots)$	The median of those absolute distances
1.4826	A constant that makes the result equivalent to a standard deviation when the noise is bell-shaped (Gaussian). This is a well-known statistics result.

The ordinary average is pulled by outliers. If five anchors have 2 cm residuals and one has 200 cm (NLOS), the average is about 35 cm — way above the typical 2 cm. The median ignores the outlier and returns 2 cm. We use the median to judge which anchors are normal and which are suspicious.

Any anchor with a residual more than $K = 2.5$ times this robust $\hat{\sigma}$ is excluded and the solve repeats with the remaining anchors.

4.5 How Certain Are We? (Position Covariance)

After the solve, we want to know how uncertain the position estimate is. The answer comes from the curvature of the cost function:

$$\mathbf{P}_{\text{pos}} \approx \sigma_{\text{base}}^2 (\mathbf{J}^\top \mathbf{W} \mathbf{J})^{-1} \quad (12)$$

$\mathbf{P}_{\text{pos}}$	The position <i>covariance matrix</i> . Its diagonal entries are the variance (spread ²) of the error in x and y (and z).
σ_{base}^2	Baseline ranging variance
$(\mathbf{J}^\top \mathbf{W} \mathbf{J})^{-1}$	The inverse of the curvature matrix. When all anchors surround the tag evenly, this is small (good geometry). When anchors are all in a line, this is large in the perpendicular direction (bad geometry).

This matrix is passed to the Kalman filter in the next step.

4.6 Solving for a Timing Bias Too (GPS-Style)

If we have $N \geq d + 2$ anchors (at least 4 in 2D, 5 in 3D), we can simultaneously solve for an unknown timing offset δ that shifts all measurements by the same amount — exactly how GPS receivers solve for their clock error:

$$\hat{d}_i = \|\mathbf{x} - \mathbf{a}_i\| + \delta + e_i \quad (13)$$

The only change to the solver is that the Jacobian gains an extra column of ones (because changing δ by 1 shifts every residual by 1 equally):

$$\tilde{\mathbf{J}}_i = [\mathbf{J}_i, 1] \quad (14)$$

Everything else stays the same.

With 3 anchors in 2D, multilateration finds (x, y) in 2 unknowns from 3 equations — one spare for checking consistency. With 4 anchors, we get a third spare which lets us also solve for the timing bias δ .

5 Step 4 — Smoothing Position Over Time (EKF / MHE)

The multilateration step gives a position every time a ranging sweep completes (about 3 times per second). That position is noisy — it jumps around even when the tag is stationary. We want to smooth it using knowledge that *people and robots do not teleport*. They move continuously, with some speed and acceleration. A **filter** uses this physics knowledge to reject noise and fill in the gaps.

5.1 The Extended Kalman Filter (EKF)

Imagine a navigator on a ship. Every few minutes they get a GPS position (noisy). Between GPS fixes, they use the ship's speed and heading (from a compass and speedometer) to estimate where the ship is now. When the next GPS fix arrives, they combine the physics-based estimate with the new measurement, trusting each one according to how reliable it has been. The Kalman filter does exactly this, but with maths.

5.1.1 The State Vector

The Kalman filter tracks everything it needs to know in a single column vector called the **state**:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{a} \\ \delta \end{bmatrix} \quad (15)$$

$\mathbf{p}$ Position — e.g. $[x, y]$ in metres in 2D
 $\mathbf{v}$ Velocity — e.g. $[v_x, v_y]$ in m/s (how fast position is changing)
 $\mathbf{a}$ Acceleration — e.g. $[a_x, a_y]$ in m/s² (how fast velocity is changing)
 δ Range bias — the common clock offset shared by all anchor measurements

The filter also keeps track of how uncertain it is about each of these quantities, stored in a matrix $\mathbf{P}$ (the covariance matrix).

5.1.2 Step A: Predict (Physics)

Between measurements, the filter predicts where the tag should be now, based on where it was and how fast it was moving. This is just Newton's kinematics:

$$\begin{aligned} \text{new position} &= \text{old position} + \text{velocity} \times \Delta t + \frac{1}{2} \times \text{acceleration} \times \Delta t^2 \\ \text{new velocity} &= \text{old velocity} + \text{acceleration} \times \Delta t \end{aligned}$$

Written for all state variables at once using a transition matrix $\mathbf{F}$:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F} \hat{\mathbf{x}}_{k-1|k-1} \quad (16)$$

$\hat{\mathbf{x}}_{k k-1}$	State estimate at time k , <i>predicted</i> using data up to time $k - 1$
$\hat{\mathbf{x}}_{k-1 k-1}$	State estimate at time $k - 1$, <i>after</i> incorporating the measurement at $k - 1$
$\mathbf{F}$	Transition matrix — encodes the kinematic equations above
Δt	Time between measurements

The matrix $\mathbf{F}$ (shown in the technical document) is just the kinematics equations packed into matrix form. Its top-left block reads: new x = old $x \times 1$ + old $v_x \times \Delta t$ + old $a_x \times \Delta t^2/2$.

The prediction uncertainty grows:

$$\mathbf{P}_{k|k-1} = \mathbf{F} \mathbf{P}_{k-1|k-1} \mathbf{F}^\top + \mathbf{Q} \quad (17)$$

$\mathbf{Q}$ is the **process noise** matrix. It represents the fact that the tag might change direction or speed in ways the filter cannot predict (e.g. the person suddenly stops). Larger $\mathbf{Q}$ = the filter knows it cannot predict well = it gives new measurements more weight.

5.1.3 Step B: Update (Measurement)

A new UWB measurement $\mathbf{z}_k$ arrives. The filter combines prediction and measurement:

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R}_k \quad (\text{total uncertainty in the predicted measurement}) \quad (18)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \mathbf{S}_k^{-1} \quad (\text{Kalman gain}) \quad (19)$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (\text{corrected estimate}) \quad (20)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (\text{uncertainty reduced after update}) \quad (21)$$

$\mathbf{H}_k$	Measurement Jacobian — how does the measurement depend on the state?
$\mathbf{R}_k$	Measurement noise covariance (= $\mathbf{P}_{\text{pos}}$ from equation 12)
$\mathbf{S}_k$	Innovation covariance — total uncertainty about what measurement to expect
$\mathbf{K}_k$	Kalman gain — the weight given to the measurement correction
$\mathbf{z}_k - h(\hat{\mathbf{x}}_{k k-1})$	“Innovation”: difference between what we measured and what we predicted
$h(\cdot)$	The function that converts a state into a predicted measurement
	For anchor i : $h_i(\mathbf{x}) = \ \mathbf{p} - \mathbf{a}_i\ + \delta$

$\mathbf{K}$ is automatically computed from two uncertainties:

- $\mathbf{P}_{k|k-1}$ (prediction uncertainty): if this is large, the filter is not confident in its prediction, so it will strongly correct toward the measurement.
- $\mathbf{R}_k$ (measurement uncertainty): if this is large, the measurement is noisy, so the filter barely corrects.

If the prediction is perfect and the measurement is noisy, $\mathbf{K} \approx 0$ (ignore the measurement). If the prediction is useless and the measurement is perfect, $\mathbf{K} \approx \mathbf{H}^{-1}$ (jump to the measurement). In practice it is always somewhere between.

The measurement model for anchor i in tight coupling is:

$$h_i(\mathbf{x}) = \|\mathbf{p} - \mathbf{a}_i\| + \delta \quad (22)$$

and its Jacobian (how does this measurement change if the state changes?):

$$\mathbf{H}_i = \begin{bmatrix} \frac{(\mathbf{p} - \mathbf{a}_i)^\top}{\|\mathbf{p} - \mathbf{a}_i\|}, & \underbrace{\mathbf{0}}_{\text{vel/acc terms}}, & 1 \end{bmatrix} \quad (23)$$

The first part is the unit vector from anchor i to the tag. It says: “moving 1 metre toward anchor i reduces the range to anchor i by 1 metre.” The final 1 is for the bias δ : increasing the bias by 1 increases all ranges by 1.

5.2 Moving Horizon Estimator (MHE)

Instead of updating one step at a time, the MHE keeps the last N measurements and solves one big optimisation problem over all of them simultaneously. Think of it like a detective who reviews all the evidence from the last 6 crime scenes together, rather than updating their theory one crime at a time. This is slower to compute but allows hard constraints (“the suspect cannot be outside the building”).

The MHE minimises this objective over the window $\{\mathbf{x}_{k-N+1}, \dots, \mathbf{x}_k\}$:

$$\min \underbrace{\|\mathbf{x}_{k-N+1} - \bar{\mathbf{x}}_{k-N+1}\|_{\mathbf{P}_{\text{arr}}}^2}_{\text{Term 1: Arrival cost (memory of history)}} + \underbrace{\sum_t \|\mathbf{x}_t - \mathbf{F}\mathbf{x}_{t-1}\|_{\mathbf{Q}}^2}_{\text{Term 2: Physics penalty (penalise unphysical jumps)}} + \underbrace{\sum_{t,i} \rho_\delta(\|\mathbf{p}_t - \mathbf{a}_i\| - \hat{d}_{t,i})}_{\text{Term 3: Range fit (fit the UWB measurements)}} \quad (24)$$

$\ \mathbf{v}\ _{\mathbf{M}}^2$	A weighted squared length: $\mathbf{v}^\top \mathbf{M} \mathbf{v}$ (penalises directions $\mathbf{M}$ says matter)
$\bar{\mathbf{x}}_{k-N+1}$	Prior estimate at the start of the window (from previous window’s solution)
$\mathbf{P}_{\text{arr}}$	Uncertainty of that prior estimate (arrival covariance)
$\mathbf{F}\mathbf{x}_{t-1}$	Where physics says $\mathbf{x}_t$ should be, given $\mathbf{x}_{t-1}$
$\rho_\delta(\cdot)$	Huber loss (explained below)
$\hat{d}_{t,i}$	UWB range measurement to anchor i at time t

The Huber loss ρ_δ — tolerating bad measurements:

$$\rho_\delta(e) = \begin{cases} \frac{1}{2}e^2 & \text{if } |e| \leq \delta \quad (\text{normal error: quadratic penalty}) \\ \delta(|e| - \frac{\delta}{2}) & \text{if } |e| > \delta \quad (\text{large error: linear penalty}) \end{cases} \quad (25)$$

Ordinary squared error (e^2) treats a 1 m NLOS error as $1/0.05^2 = 400$ times worse than a normal 5 cm error. One bad anchor dominates everything. The Huber loss switches to linear growth for large errors, so the 1 m outlier only contributes about 13 times a normal 5 cm error — a much fairer penalty that does not let one bad anchor ruin the solution.

Error $ e $	Squared: e^2	Huber ($\delta = 0.15$ m)	Ratio
0.05 m	0.0025	0.00125	$1\times$ (baseline)
0.15 m	0.0225	0.01125	$9\times$
0.30 m	0.0900	0.01688	$13\times$ (Huber caps growth)
1.00 m	1.0000	0.13500	$108\times$ vs $400\times$ for squared

Hard constraints prevent the MHE from placing the tag outside the room or moving it faster than a person can walk:

$$\mathbf{p}_{\min} \leq \mathbf{p}_t \leq \mathbf{p}_{\max} \quad (\text{room boundary}) \quad (26)$$

$$\|\mathbf{v}_t\| \leq v_{\max} \quad (\text{speed limit, e.g. 2 m/s for walking}) \quad (27)$$

EKF = fast, simple, real-time. MHE = slower, handles hard constraints and outliers better. Use EKF first; upgrade to MHE for hostile environments.

6 Step 5 — Adding Orientation (IMU Fusion)

UWB tells us *where* the tag is (position), but not *which way it is pointing* (orientation). A BNO085 IMU (Inertial Measurement Unit) measures rotation and acceleration 100 times per second. Adding it gives us:

1. **Roll, Pitch, Yaw** — the three angles that describe orientation.
2. **30 position estimates per UWB sweep** — by integrating acceleration between UWB fixes.

6.1 Describing Orientation — Why Quaternions?

Euler angles (roll, pitch, yaw): easy to understand, but have a bug called *gimbal lock*. When you pitch up 90 degrees, roll and yaw point in the same direction and you lose a degree of freedom — the maths breaks down.

Quaternions: four numbers instead of three, with one constraint. No gimbal lock, no singularities. The BNO085 outputs quaternions directly, so we never experience gimbal lock.

A quaternion is written $\mathbf{q} = [q_w, q_x, q_y, q_z]$ with the constraint $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$ (unit length). The geometric meaning: any rotation can be described as “rotate by angle θ about a unit axis $\hat{n}$ ”, encoded as:

$$\begin{aligned} q_w &= \cos(\theta/2) && \text{The “how much” part} \\ q_x &= \hat{n}_x \sin(\theta/2) && \text{The } x\text{-component of the axis, scaled} \\ q_y &= \hat{n}_y \sin(\theta/2) && \text{The } y\text{-component of the axis, scaled} \\ q_z &= \hat{n}_z \sin(\theta/2) && \text{The } z\text{-component of the axis, scaled} \end{aligned}$$

Axis = $[0, 0, 1]$ (straight up, the Z axis). Angle $\theta = 90^\circ$.

$$q_w = \cos(45^\circ) = 0.707, \quad q_x = 0, \quad q_y = 0, \quad q_z = \sin(45^\circ) = 0.707$$

So $\mathbf{q} = [0.707, 0, 0, 0.707]$.

6.2 The Rotation Matrix

To rotate a vector from the body frame (tag’s perspective) into the world frame (room’s perspective), we use:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \quad (28)$$

Each entry is a quadratic combination of q_w, q_x, q_y, q_z . If the tag accelerates forward (body frame), multiply $\mathbf{R}$ by that acceleration vector to get the world-frame direction of the acceleration. This is needed to integrate IMU acceleration into world-frame position.

6.3 How Orientation Updates (Quaternion Kinematics)

The gyroscope measures angular rate $\boldsymbol{\omega}^b = [\omega_x, \omega_y, \omega_z]$ in rad/s. The continuous-time update rule is:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}^b \end{bmatrix} \quad (29)$$

$\dot{\mathbf{q}}$	Time derivative of the quaternion (how fast it is changing)
$\otimes$	Hamilton product (the rule for multiplying two quaternions, similar to cross product)
$[0, \boldsymbol{\omega}^b]$	The angular rate packed as a pure quaternion (zero scalar part)
$\frac{1}{2}$	A factor that comes from the chain rule applied to the angle-halving in quaternions

For a finite time step Δt (e.g. 10 ms at 100 Hz), the discrete update is:

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \begin{bmatrix} \cos\left(\frac{\Delta t}{2} \|\boldsymbol{\omega}_k^b\|\right) \\ \frac{\boldsymbol{\omega}_k^b}{\|\boldsymbol{\omega}_k^b\|} \sin\left(\frac{\Delta t}{2} \|\boldsymbol{\omega}_k^b\|\right) \end{bmatrix} \quad (30)$$

The big bracket on the right is a quaternion that represents “rotate by $\|\boldsymbol{\omega}^b\| \cdot \Delta t$ radians about the axis $\boldsymbol{\omega}^b / \|\boldsymbol{\omega}^b\|$.” Multiplying (Hamilton product $\otimes$) the current orientation quaternion by this rotation quaternion rotates it to the new orientation. This is exactly analogous to multiplying rotation matrices, but numerically more stable. The BNO085 chip handles all of this internally and outputs the result directly.

6.4 Position Propagation from IMU

The BNO085 also reports linear acceleration $\mathbf{a}^b$ (gravity already removed) in the body frame. Steps:

1. Rotate to world frame: $\mathbf{a}^w = \mathbf{R}(\mathbf{q}) \mathbf{a}^b$

2. Integrate (Newton's equations):

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k^w \Delta t^2 \quad (31)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k^w \Delta t \quad (32)$$

This runs at 100 Hz. Every UWB sweep (at 3 Hz) provides 33 intermediate positions, making the output much smoother. The UWB measurement then corrects any drift accumulated during those 33 IMU steps.

6.5 Extracting Roll, Pitch, Yaw from a Quaternion

Roll ϕ : tilting left/right (like an airplane banking). **Pitch** θ : nose up or down. **Yaw** ψ : turning left or right (like a compass bearing). These three angles together fully describe orientation.

From a unit quaternion $[q_w, q_x, q_y, q_z]$:

$$\phi = \text{atan2}\left(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)\right) \quad (33)$$

$$\theta = \arcsin(2(q_w q_y - q_z q_x)) \quad (34)$$

$$\psi = \text{atan2}\left(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)\right) \quad (35)$$

$\text{atan2}(y, x)$ The 4-quadrant arctangent. Returns the angle whose tangent is y/x , choosing the correct quadrant based on the signs of y and x .
Range: -180° to $+180^\circ$.

$\arcsin(x)$ Inverse sine. Returns the angle whose sine is x .
Range: -90° to $+90^\circ$ (this is why pitch is limited to $\pm 90^\circ$).

The BNO085 ARVR Stabilised Rotation Vector mode fuses accelerometer + gyroscope + magnetometer internally at 400 Hz. The magnetometer provides a compass heading (absolute yaw reference), preventing the yaw angle from drifting over time as it would with a gyroscope alone.

7 Step 6 — Finding Anchor Positions Automatically (MDS)

Normally, the anchor positions must be measured by hand with a tape measure and typed into the configuration file. Even small errors (1–2 cm) directly bias the position output. Anchor self-survey replaces the tape measure: the anchors range to *each other*, and a mathematical technique called Multidimensional Scaling (MDS) reconstructs their 2D or 3D layout automatically.

You are given a table of road distances between 4 cities but no map. You want to reconstruct the map. MDS is the procedure that does exactly this — from pairwise distances, it recovers the 2D coordinates of each point.

7.1 Collecting Pairwise Distances

With N anchors, there are $\binom{N}{2} = N(N-1)/2$ unique pairs. For 4 anchors: $4 \times 3/2 = 6$ pairs. Each anchor temporarily acts as a tag and ranges to every other anchor. Each pair's distance is averaged over $M = 100$ measurements. By the statistics rule for averaging: the uncertainty reduces from σ_d per measurement to $\sigma_d/\sqrt{M}$.

Each single UWB measurement has noise $\sigma_d \approx 5$ cm. After averaging 100 measurements:

$$\text{uncertainty of mean} = \frac{5 \text{ cm}}{\sqrt{100}} = \frac{5}{10} = 0.5 \text{ cm}$$

We get sub-centimetre anchor position accuracy for free, just by averaging more measurements.

7.2 Step 1: The Squared-Distance Matrix

Arrange all pairwise distances in a symmetric $N \times N$ matrix (rows and columns are anchors), and square each entry:

$$D_{ij}^{(2)} = \hat{d}_{ij}^2$$

The diagonal is zero (each anchor is 0 metres from itself). This matrix encodes all the geometric information.

7.3 Step 2: Double-Centering

Apply the centering matrix $\mathbf{J} = \mathbf{I}_N - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$, where $\mathbf{I}_N$ is the identity matrix and $\mathbf{1}$ is a column of ones:

$$\mathbf{B} = -\frac{1}{2}\mathbf{J}\mathbf{D}^{(2)}\mathbf{J} \quad (36)$$

Consider four anchor positions packed as rows of a matrix $\mathbf{X}$. The squared distance between anchor i and anchor j expands as:

$$d_{ij}^2 = \|\mathbf{x}_i - \mathbf{x}_j\|^2 = \|\mathbf{x}_i\|^2 - 2\mathbf{x}_i \cdot \mathbf{x}_j + \|\mathbf{x}_j\|^2$$

The terms $\|\mathbf{x}_i\|^2$ and $\|\mathbf{x}_j\|^2$ depend only on how far each anchor is from the origin — which is an arbitrary choice we have not made yet. The double-centering operation cancels these terms and leaves only $B_{ij} = \mathbf{x}_i \cdot \mathbf{x}_j$ (the dot product between positions). In matrix form: $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$. We have converted distances (which depend on the origin) into dot products (which describe shape, independent of origin).

7.4 Step 3: Eigendecomposition

Decompose $\mathbf{B}$ into eigenvalues and eigenvectors:

$$\mathbf{B} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$$

$\mathbf{V}$ Matrix of eigenvectors (each column is an eigenvector)
 $\mathbf{\Lambda}$ Diagonal matrix of eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N$
 $\mathbf{V}^\top$ Transpose of $\mathbf{V}$

An eigenvector of a matrix is a special direction that the matrix does not rotate — it only stretches or shrinks it. The eigenvalue is the stretch factor. For our matrix $\mathbf{B}$, which encodes a 2D shape, there are exactly 2 large eigenvalues (one for each spatial dimension). All others are near zero. The two eigenvectors corresponding to the two largest eigenvalues point along the two axes of the shape.

Take the top d eigenvalues (2 for 2D, 3 for 3D) and their eigenvectors:

$$\hat{\mathbf{X}} = \mathbf{V}_d \mathbf{\Lambda}_d^{1/2} \quad (37)$$

$\mathbf{V}_d$ The d eigenvectors with the largest eigenvalues, as columns
 $\mathbf{\Lambda}_d^{1/2}$ A diagonal matrix with $\sqrt{\lambda_1}, \dots, \sqrt{\lambda_d}$ on the diagonal
 (square root because $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$ and we want $\mathbf{X}$, not $\mathbf{X}\mathbf{X}^\top$)
 $\hat{\mathbf{X}}$ The recovered anchor positions: each row is one anchor's (x, y) or (x, y, z)

True positions: $A_1 = (0, 0)$, $A_2 = (3, 0)$, $A_3 = (1.5, 2.6)$ metres. All sides = 3 m.

$$\mathbf{D}^{(2)} = \begin{bmatrix} 0 & 9 & 9 \\ 9 & 0 & 9 \\ 9 & 9 & 0 \end{bmatrix}$$

After double-centering and eigendecomposition, taking the 2 largest eigenvectors and scaling by $\sqrt{\lambda}$, we recover a layout matching the triangle (up to rotation and reflection). After frame fixation (below), we get back the original coordinates.

7.5 Step 4: Fix the Coordinate Frame

MDS recovers the shape but not its orientation or position (there is no “north” in the math). Three rules fix a unique frame:

1. **Move anchor 1 to the origin.** Subtract $\hat{x}_1$ from every row of $\hat{\mathbf{X}}$.
2. **Rotate so anchor 2 is on the positive X axis.** Compute the angle of $\hat{x}_2$ and rotate all points by $-\text{that angle}$.
3. **Flip if needed.** If anchor 3 ends up below the X axis (negative y), negate all y coordinates.

The resulting layout is saved to `config/anchors.json` and used immediately. No tape measure needed.

7.6 How Accurate Is It?

Each pairwise distance is the mean of 100 measurements. The averaging reduces noise by a factor of $\sqrt{100} = 10$. MDS propagates this to anchor position uncertainty of approximately

$\sigma_d/\sqrt{M}$ per coordinate:

$$\text{Position accuracy} \approx \frac{5 \text{ cm}}{\sqrt{100}} = 0.5 \text{ cm per anchor}$$

This is well within the target of ≤ 2 cm for anchor self-survey.

All the maths in this section uses only: squared distances, averages, and eigenvalues. No GPS, no cameras, no manual measurement.

8 How All the Pieces Fit Together

§	Step	Input	Output
2	DS-TWR ranging	6 timestamps (per anchor)	Distance $\hat{d}_i$ to each anchor
3	NLOS detection	FP_AMPL + RXPACC + CIR_PWR	NLOS score, corrected $\hat{d}_i$
4	Multilateration	N corrected distances + anchor positions	Position estimate $\hat{\mathbf{p}}$ + covariance
5	EKF / MHE	New position every sweep	Smoothed position $\hat{\mathbf{x}}$ at each step
6	IMU fusion	Quaternion + linear accel at 100 Hz	Roll/pitch/yaw + high-rate positions
7	MDS self-survey	Anchor-to-anchor distances	Anchor positions in <code>anchors.json</code>

The six phases of improvement map directly to these steps:

Phase	Improves	Target accuracy
0 (done)	MATLAB robustness	Stable 3–10 cm baseline
1	PRF + calibration + self-survey	2–6 cm
2	NLOS weighting	1–4 cm
3	EKF/MHE	1–3 cm
4	3D + IMU fusion + RPY	1–3 cm XY, 2–5 cm Z, 3°–5° RPY
5–6	Hardening + multi-tag	Same accuracy, more robust